

Ch_17 LangGraph

LangGraph의 동작원리에 대한 설명

[테디노트-이경록-랭체인밋업2024-Q2-발표자료-V05.pdf](#)

I. 핵심 기능

A) LangGraph에서 자주 쓰는 Python 문법 핵심 정리

TypedDict

: 구조가 정해진 dict (타입 안전한 딕셔너리)

1. 특징

- `dict`: 타입 검사 X (런타임에만 동작)
- `TypedDict`: 정적 타입 검사 가능 (IDE, mypy 등에서 오류 체크)

2. 왜 사용하는가?

- 타입 안정성 ↑ (버그 미리 방지)
- 코드 가독성 ↑ (구조 명확)
- 자동완성 & 타입 힌트 지원
- 코드 자체가 문서 역할

3. 핵심 포인트

- 각 key별 타입 지정 가능

- 정의되지 않은 key 추가 X
- 타입 틀리면 오류 발생

Annotated

: 타입에 추가 설명(메타데이터)을 붙이는 문법

1. 역할

- 타입 + 추가 정보 같이 표현
- 코드 설명 + 문서화 효과
- 유효성 검사 (Pydantic 등과 함께 사용)

2. 예시 개념

```
name: Annotated[str, "이름"]
age: Annotated[int, "나이"]
```

3. 핵심 포인트!!!

- 런타임 영향 없음 (설명용)
- IDE / 프레임워크에서 활용
- LangGraph에서 특별 동작 정의에 사용됨

add_messages

: 메시지 리스트를 누적/병합하는 함수

1. 역할

- 메시지 리스트를 **append** 방식으로 유지
- 기존 상태를 덮어쓰지 않고 추가

2. 동작 방식

- 같은 ID → 덮어쓰기

- 다른 ID → 추가
- 두 리스트(left, right) 병합

핵심 포인트!!!

- 상태 유지 (chat history 관리 핵심)
- Annotated와 함께 사용됨

한줄 정리

- `TypedDict` → 구조 정의된 안전한 dict
- `Annotated` → 타입 + 설명/메타데이터
- `add_messages` → 메시지 누적/병합 로직

B) LangGraph 챗봇 구축 핵심 정리

1. LangGraph 핵심 개념

: LangGraph = 상태 기반 그래프(State Machine)

2. 구성 요소

- **State**: 데이터(메모리, context)
- **Node**: 실행 함수 (작업 단위)
- **Edge**: 흐름 연결 (어디로 갈지 결정)

상태를 들고 → 함수 실행 → 다음 단계로 이동 구조

2. 전체 흐름

① State 정의

```
class State(TypedDict):
    messages: Annotated[list, add_messages]
```

의미

- 메시지를 저장하는 공간
- `add_messages` → 메시지 누적

② Node 정의

```
def chatbot(state: State):  
    return {"messages": [llm.invoke(state["messages"])]}
```

의미

- 하나의 작업 단위 (LLM 호출 등)
- 입력: state
- 출력: 업데이트된 state

→ Node = "기능 함수"

③ Graph 생성

```
graph = StateGraph(State)
```

의미

- 전체 흐름 틀 생성
- 상태 기반 실행 구조 정의

④ Node 추가

```
graph.add_node("chatbot", chatbot)
```

의미

- 사용할 기능 등록

⑤ Edge 연결

```
graph.add_edge(START, "chatbot")  
graph.add_edge("chatbot", END)
```

의미

- 실행 흐름 정의
- START → chatbot → END

⑥ 실행 (compile)

```
app=graph.compile()
```

의미

- 실제 실행 가능한 구조로 변환

핵심 구조

```
[START]
  ↓
[Node (LLM 호출)]
  ↓
[END]
```

⇒ 상태(State)를 계속 업데이트하면서 흐름 진행

핵심 포인트!!!

1. State 중심 설계

- 모든 데이터는 state에 저장
- 이전 결과 계속 활용 가능

2. Node = 함수

- 그냥 Python 함수
- 입력: state / 출력: state

3. Edge = 흐름 제어

- 조건 분기 가능
- 반복 구조 가능

4. 메시지 기반 구조

- chat history를 계속 누적
- 대화형 시스템에 최적화

LangGraph = 상태(State)를 들고 Node를 따라 흐르는 "AI 워크플로우 그래프"

C) LangGraph Agent 구축 핵심 정리

1. Agent 개념

: LLM + Tool(도구) = Agent

- LLM이 모르는 질문 → 도구 호출해서 해결
- 예: 웹 검색, DB 조회 등

2. 전체 구조

```
graph TD
    A[User 질문] --> B[LLM chatbot]
    B -- "(필요하면)" --> C[Tool 호출]
    C --> D[다시 LLM]
    D --> E[응답]
```

: LLM ↔ Tool을 오가며 답 생성

3. Tool (도구)

: 외부 기능을 수행하는 역할

i) 예시

- 웹 검색 (Tavily)
- API 호출
- DB 조회

ii) 특징

- LLM이 필요할 때만 호출
- 결과를 다시 LLM이 사용

4. 핵심 구현 흐름

① Tool 생성

```
tool= TavilySearch(max_results=3)
tools= [tool]
```

② LLM + Tool 결합

```
llm_with_tools=llm.bind_tools(tools)
```

- LLM이 직접 Tool 사용할 수 있음

③ Chatbot Node

```
def chatbot(state):  
    return {"messages": [llm_with_tools.invoke(state["message  
s"])]}
```

- LLM 실행 (필요 시 tool 호출)

④ Tool Node

- 실제 도구 실행 담당
- LLM이 tool 호출 요청 → 여기서 실행
- 결과를 다시 message로 반환

5. Conditional Edge

조건에 따라 흐름 분기

```
if tool_calls 존재:  
    → tools노드  
else:  
    →END
```

의미

- 도구 필요 → 실행
- 필요 없음 → 종료

→ 즉, if문 기반 흐름 제어

6. 전체 흐름 정리

```
START  
↓  
chatbot (LLM)
```

↓
[조건 판단]
├─ tool 필요 → tools → 다시 chatbot
└─ 필요 없음 → END

⇒ 반복 가능한 루프 구조

핵심 포인트!!!

1. LLM은 “판단자”

- tool 쓸지 말지 결정

2. Tool은 “행동자”

- 실제 작업 수행 (검색 등)

3. Graph는 “흐름 관리자”

- 어디로 갈지 제어

D) LangGraph Agent + Memory 핵심 정리

1. 왜 Memory가 필요한가?

기본 Agent 문제

- 이전 대화 기억 X
- 멀티턴 대화 불가능

→ 해결: **Memory** 추가

2. 핵심 개념

Checkpointing

상태(State)를 자동 저장

- 각 단계마다 state 저장
- 다시 실행하면 이어서 시작 가능

→ 대화 기록 자동 저장 + 이어서 대화 가능

3. Memory 구현 방식

MemorySaver

```
memory=MemorySaver()
```

가장 기본적인 메모리 저장 방식

- in-memory (임시 저장)
- 실제 서비스: DB 사용
 - SQLite / Postgres 등

4. 핵심 구조 변화

기존 Agent

```
질문 → LLM → 응답
```

Memory 추가 Agent

```
질문 → State 저장 → LLM → 응답 → State 저장
```

→ 매 단계마다 기억됨

5. thread_id

대화 세션 구분 키

```
config= {  
    "thread_id": "1"  
}
```

특징

- 같은 thread_id → 기억 유지
- 다른 thread_id → 새로운 대화

→ 유저별 대화 구분

6. RunnableConfig 핵심

```
RunnableConfig(  
    recursion_limit=10,
```

```
) configurable={"thread_id":"1"}
```

옵션

- `recursion_limit`: 최대 실행 단계
- `thread_id`: 대화 식별자

7. 전체 흐름

```
START
  ↓
chatbot (LLM)
  ↓
[조건]
├─ tool 필요 → tools → chatbot
└─ 종료
```

추가된 것 ⇒ 매 단계마다 **state 저장 (checkpoint)**

8. 상태 확인 (디버깅용)

```
graph.get_state(config)
```

- 저장된 메시지 확인 가능
- "지금까지 기억 뭐 있지?" 확인

핵심 포인트!!!

1. checkpointing = 기억의 핵심

- 자동 저장 + 이어서 실행

2. thread_id = 사용자 구분

- 같은 사람 → 기억 유지
- 다른 사람 → 초기화

3. Memory ≠ 단순 리스트

- LangGraph는 상태 전체를 저장

E) LangGraph Human-in-the-loop 핵심 정리

1. 개념

: Human-in-the-loop = 사람이 중간에 개입하는 구조

- AI는 완전히 신뢰 불가능
- 중요한 순간 → 사람이 판단 필요 ⇒ "AI + 사람 협업 시스템"

2. 왜 사용하는가?

- 잘못된 tool 실행 방지
- 중요한 작업 승인 필요
- 결과 검증 (안전성 확보)

예시)

- 결제 승인
- 데이터 삭제
- 중요한 검색 결과 확인

3. 핵심 기능

i) interrupt_before

- 특정 노드 실행 전에 멈춤

```
interrupt_before= ["tools"]
```

ii) 의미

- tool 실행 전에 STOP
- 사람이 확인 후 진행

자동 → 수동 승인 단계 추가

4. 동작 흐름

```
User
  ↓
chatbot (LLM)
  ↓
```

```
interrupt (일시 정지)
```

↓

인간 판단

↓

tools 실행

↓

chatbot → 응답

→ 중간에 사람이 개입

5. Memory와 결합

- checkpoint 덕분에 → 중간에서 멈춰도 다시 시작 가능

일시정지 → 재개 가능 시스템

6. 상태 확인

```
graph.get_state(config)
```

→ 현재 어디까지 진행됐는지 확인

7. Replay (되돌리기)

→ 특정 시점부터 다시 실행

- checkpoint_id 사용
- 과거 상태로 돌아가 재실행

디버깅 + 수정 가능

핵심 포인트!!!

1. interrupt = 멈춤

- 특정 단계에서 실행 중단

2. human approval = 승인

- 사람이 판단 후 진행

3. replay = 되돌리기

- 이전 상태에서 다시 실행

F) LangGraph 상태 수정 & Replay 심화 핵심 정리

1. 핵심 개념

: State를 중간에 직접 수정할 수 있음

- 자동 흐름만 따르는 게 아니라 → 사람이 상태를 바꿔서 흐름 자체를 수정 가능

2. 왜 사용하는가?

- Agent가 잘못된 판단했을 때 수정
- 다른 경로로 실행하고 싶을 때
- 결과를 바탕으로 "과거 수정" 가능

→ AI 흐름을 사람이 컨트롤하는 기능

3. 핵심 기능

i) 상태(State) 수동 업데이트

중간 상태를 직접 변경

```
update_state(...)
```

ii) 의미

- 기존 흐름 무시 가능
- 새로운 방향으로 실행

→ AI 행동을 강제로 수정

4. Replay (되돌리기) 심화

: 특정 시점으로 돌아가 다시 실행

- checkpoint 기반
- 과거 상태 → 다시 시작

→ 단순 재실행이 아니라 수정 후 재실행 가능

5. 핵심 차이

✓ 기존 Replay

- 그냥 과거로 돌아가감

✓ 이번 Replay

- 상태 수정 후 → 다시 실행
- 훨씬 강력함

6. 전체 흐름

```
실행 중
↓
중단 (interrupt)
↓
상태 수정 (update_state)
↓
replay (재실행)
↓
새로운 결과
```

핵심 포인트!!!

1. State 수정 가능

- 흐름 자체를 바꿀 수 있음

2. Replay + 수정

- 단순 되돌리기가 아님 → “다르게 다시 실행”

3. 디버깅 & 실험에 최적

- 다양한 경로 테스트 가능

G) LangGraph SubGraph (서브그래프) 핵심 정리

1. 개념

: SubGraph = 그래프 안에 또 다른 그래프

- 하나의 Node처럼 사용 가능
- 내부는 독립적인 workflow

즉, 큰 시스템을 작은 그래프로 나누는 구조

2. 왜 사용하는가?

- 복잡한 시스템 관리
- 기능별 모듈 분리
- 멀티 에이전트 구현

→ 대표 사용

→ **Multi-Agent 시스템**

3. 핵심 구조

Main Graph

↓

SubGraph (하나의 Node처럼 동작)

↓

내부 여러 Node 실행

→ Node 안에 또 다른 Graph

4. 중요한 포인트: State 공유 방식

Case 1: State 공유 (같은 schema)

- 같은 key 사용 (ex. messages)
- 그냥 SubGraph를 Node처럼 추가

→ 가장 간단한 방식

Case 2: State 다름 (다른 schema)

- 변환 필요

Main State → SubGraph State → 다시 Main State

→ 중간에 변환 함수 필요

→ 데이터 구조 다르면 반드시 변환해야 함

5. 핵심 흐름

START

↓

```
graph TD
    A[Main Graph] --> B[SubGraph 호출]
    B --> C[SubGraph 내부 실행]
    C --> D[결과 반환]
    D --> E[Main Graph 계속 진행]
```

핵심 포인트!!!

1. SubGraph = 재사용 가능한 모듈

- 기능 단위로 분리 가능

2. State 공유가 핵심

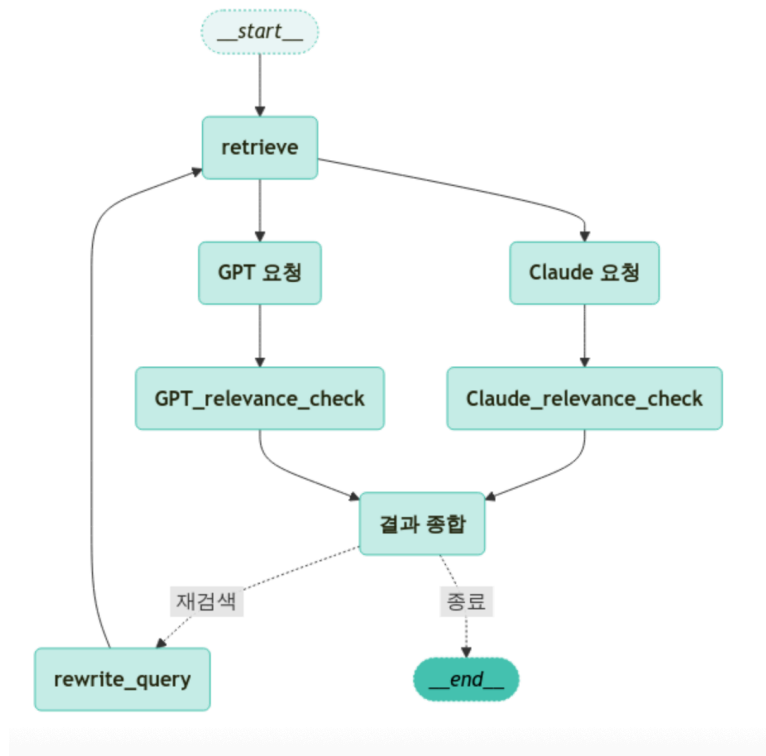
- 같은 구조면 편함
- 다르면 변환 필요

3. 멀티 에이전트 핵심 구조

- Agent들을 각각 SubGraph로 구성 가능

II. 구조 설계

A) LangGraph 기본 그래프 생성



1. 개념

: LangGraph의 가장 기본 구조 만드는 과정

- 그래프 기반 AI 흐름을 처음부터 구성하는 단계

2. 생성 단계

1. State 정의
2. Node 정의
3. Graph 생성
4. Graph 컴파일
5. (선택) 시각화

이 순서대로 진행함!

3. 핵심 요소

i) State

- 데이터 저장 (question, answer 등)

ii) Node

- 실행 함수 (검색, LLM 호출 등)

iii) Edge

- 흐름 연결 (어디로 갈지 결정)

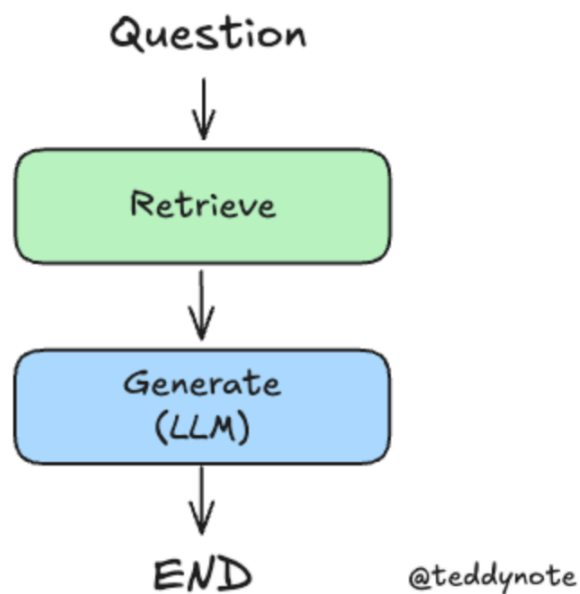
4. 특징

- 여러 Node 연결해서 복잡한 흐름 구성 가능
- 조건부 분기(Conditional Edge) 가능
- 다양한 처리 단계 추가 가능

핵심 포인트!!!

- Graph = 흐름 구조
- Node = 기능
- State = 데이터

B) Naive RAG (기본 RAG 구조)



1. 개념

: 가장 기본적인 RAG 구조 (검색 + 생성)

- 외부 문서에서 정보 가져오고 → LLM이 답 생성

2. 핵심 흐름

질문 → 문서 검색 → LLM → 답변

→ 가장 단순한 Retrieval 구조

3. LangGraph에서 특징

- Retriever와 Chain을 분리해서 사용 → 각 Node에서 세부 처리 가능

4. 구성 요소

i) Retriever

- 관련 문서 검색

ii) LLM (Chain)

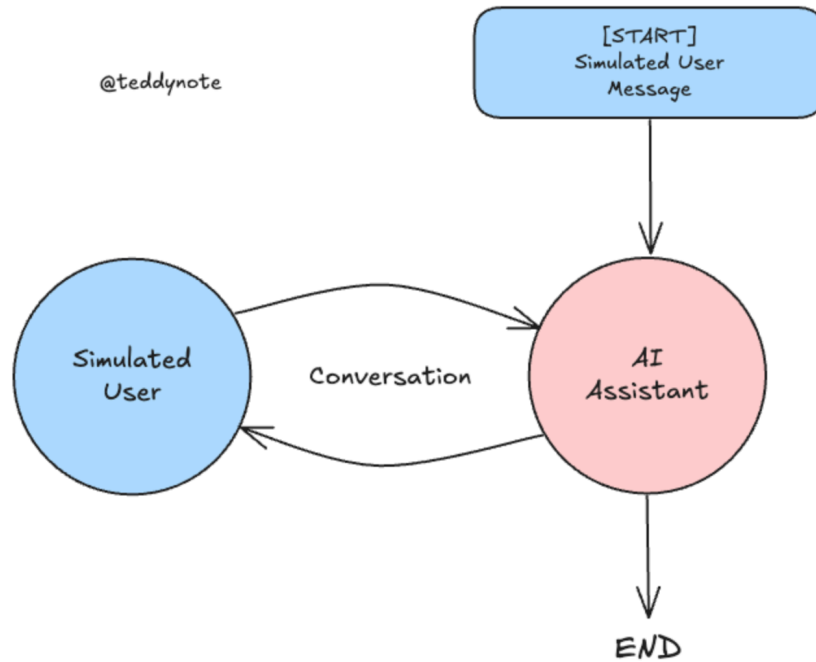
- 검색 결과 기반으로 답 생성

핵심 포인트!!!

- 가장 기본적인 RAG 구조
- 검색 결과 그대로 사용 (검증 없음)
- 이후 발전: Self-RAG, CRAG 등

III. Use Cases

A) LangGraph 대화 시뮬레이션 (Use Case)



1. 개념

: 가상 사용자(Simulated User)로 대화 테스트하는 구조

- 실제 사람이 없어도 → 챗봇과 자동으로 대화 진행 가능

2. 왜 사용하는가?

- 챗봇 성능 테스트
- 반복 테스트 자동화
- 다양한 시나리오 검증

** 수동 테스트 → 시간 많이 듦

** 시뮬레이션 → 자동 반복 가능

3. 핵심 구조

가상 사용자 → 챗봇 → 응답 → 다시 사용자

: 사람 없이도 대화 루프 생성

4. 구성 요소

i) State

- messages 저장 (대화 기록)

ii) 역할 분리

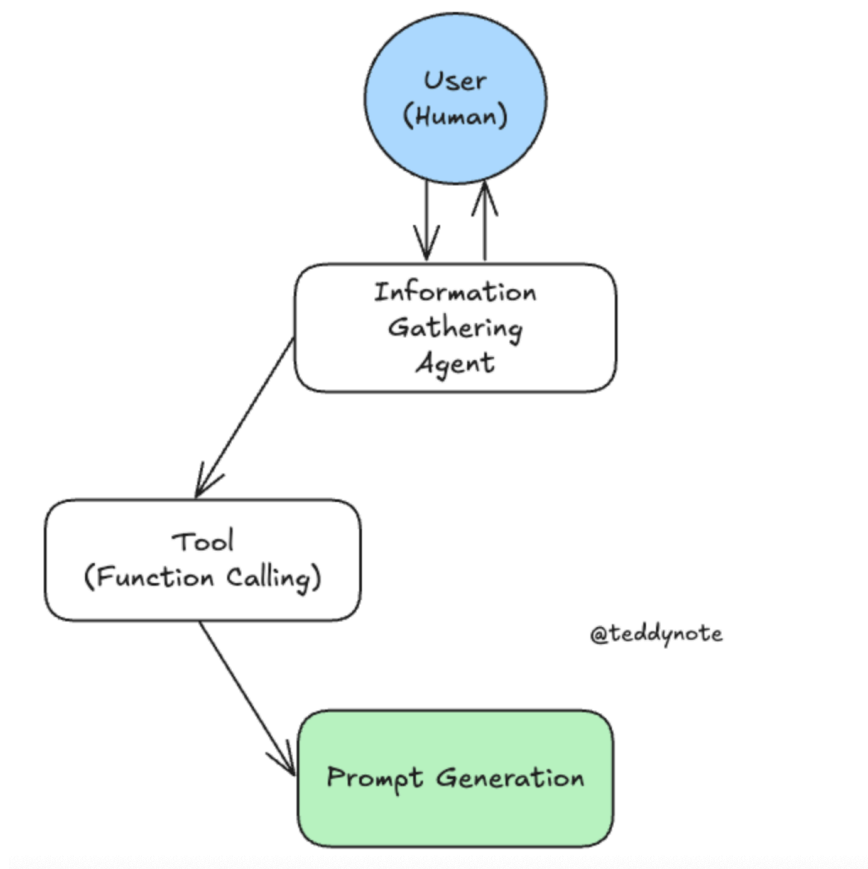
- 상담사(챗봇)
- 고객(가상 사용자)

→ 서로 역할 나눠서 대화 진행

핵심 포인트!!!

- 테스트용 Agent 구조
- 자동 대화 생성
- 반복 실험 가능

B) 사용자 요구 기반 메타 프롬프트 생성 Agent



1. 개념

: 사용자의 요구사항을 받아서 "프롬프트를 대신 만들어주는 Agent"

- 사람이 직접 프롬프트 안 짜도 됨
- AI가 최적 프롬프트 생성 (wikidocs.net)

2. 핵심 흐름

요구사항 입력 → 정보 수집 → 프롬프트 생성 → 수정 → 완료

→ 두 단계 구조

1. 정보 수집
2. 프롬프트 생성

3. 핵심 단계

i) 정보 수집 (info)

- 목표 (objective)
- 변수 (variables)
- 제약 (constraints)
- 요구사항 (requirements)

→ 부족하면 계속 질문

ii) 프롬프트 생성 (prompt)

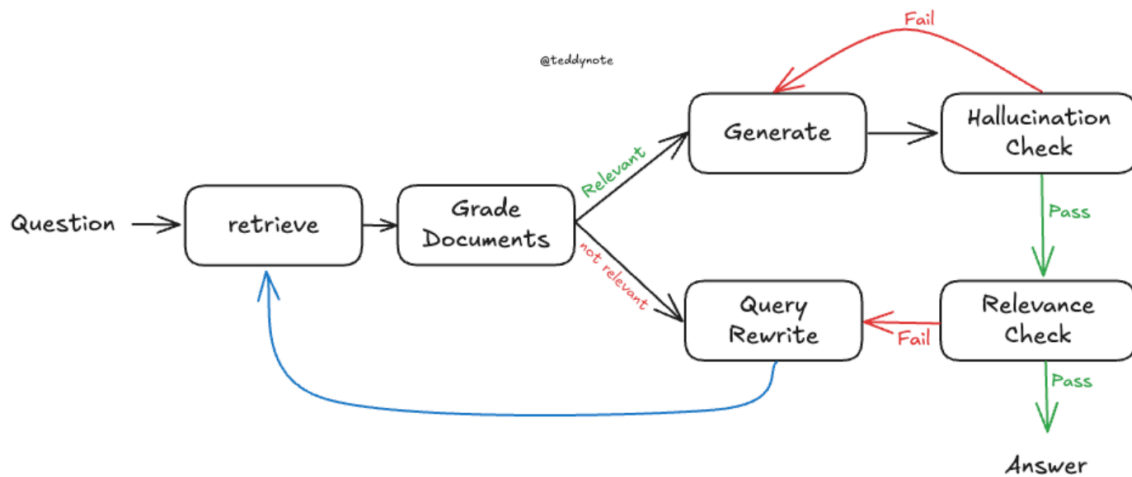
- 수집된 정보 기반으로 → **최적 시스템 프롬프트 생성**

→ 메타 프롬프트 사용

핵심 포인트!!!

- 상태 2개로 나뉨 (info / prompt)
 - LLM이 상태 전환 결정
 - 사용자 입력 기반으로 계속 개선
-
- 프롬프트를 "직접 쓰는 게 아니라 생성"
 - 반복적으로 개선 가능
 - 구조화된 프롬프트 생성
-

C) Self-RAG (자기 평가 RAG) 핵심 정리



1. 개념

→ Self-RAG = RAG + 자기 평가(self-check)

- 단순히 검색하고 답변 X
- 검색 + 검증 + 평가까지 수행

2. 기존 RAG vs Self-RAG

기존 RAG

검색 → LLM → 답변

Self-RAG

검색 → 평가 → 생성 → 검증 → 개선

→ 답변을 스스로 점검함

3. 핵심 기능

i) 검색 필요 여부 판단

- 검색할지 말지 결정

ii) 문서 관련성 평가

- 가져온 정보가 유용한지 확인

iii) 사실성 검증

- 답변이 근거 있는지 체크

iv) 답변 품질 평가

- 질문을 잘 해결했는지 점수화

핵심 포인트!!!

- "검색 → 생성" 끝이 아님
- 스스로 평가하고 수정하는 구조

D) SQL DB와 상호작용하는 Agent

1. 개념

: 자연어 질문 → SQL 쿼리 생성 → DB 조회 → 답변 생성

- LLM이 DB를 직접 조회해서 답함

2. 핵심 흐름

질문 → 테이블 파악 → 쿼리 생성 → 실행 → 결과 → 답변

3. 상세 단계

1. DB 테이블 구조 확인
2. 관련 테이블 선택
3. 스키마(DDL) 조회
4. SQL 쿼리 생성
5. 쿼리 검증 (오류 체크)
6. 쿼리 실행
7. 결과 기반 답변 생성

핵심 포인트!!!

- LLM이 직접 SQL 작성

- 잘못된 쿼리 → 수정 후 재실행
 - 실제 데이터 기반 답변
-

- "텍스트 → SQL → 데이터 → 답변"
- Tool 기반 Agent 구조
- 데이터 기반 정확한 응답 가능